RETRIEVAL-AUGMENTED GENERATION (RAG)

One-Page Introduction and Quick Notes

1. Introduction

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval with large language models. Instead of relying only on the knowledge stored in an LLM, RAG retrieves relevant information from an external knowledge base and provides it to the model as context before generating an answer.

2. Why RAG?

LLMs may have outdated knowledge, limited knowledge of private documents, or produce hallucinated information. RAG helps address these limitations by retrieving relevant and up-to-date information from a controlled knowledge source. It is especially useful for private, domain-specific, and frequently changing information.

3. RAG Architecture

A RAG system generally contains a **document processing layer, embedding model, vector database, retriever, prompt/context layer, and LLM**. Documents are converted into searchable representations, stored in a vector database, and retrieved when a user asks a question.

4. RAG Pipeline

Documents → **Parsing** → **Chunking** → **Embedding** → **Vector Database** → **User Query** → **Query Embedding** → **Retrieval** → **Context** → **LLM** → **Answer**. During ingestion, documents are processed and divided into smaller chunks. Each chunk is converted into an embedding and stored in a vector database. During inference, the user query is embedded and similar chunks are retrieved as context for the LLM.

5. Document Parsing and Chunking

Parsing extracts useful content from documents such as PDFs, text files, web pages, or other sources. **Chunking** divides large documents into smaller pieces so that relevant information can be retrieved efficiently. Good chunk size and overlap can improve retrieval quality and preserve important context.

6. Embeddings and Vector Database

Embeddings represent text as numerical vectors that capture semantic meaning. These vectors are stored in a **vector database** such as Qdrant, Chroma, Pinecone, or Weaviate. When a query arrives, its embedding is compared with stored embeddings to find semantically similar document chunks.

7. Retrieval

The retriever searches the vector database and returns the most relevant chunks for the user's question. Retrieval can use similarity search, maximum marginal relevance, metadata filtering, or hybrid search. The retrieved information becomes the context provided to the language model.

8. Generation

The LLM receives the user's question together with the retrieved context. It uses this information to generate a natural-language answer. A well-designed RAG prompt instructs the model to answer using the retrieved context and avoid unsupported claims.

9. Advantages

RAG can improve factual grounding, provide access to private knowledge, reduce reliance on model training data, support frequently updated information, and provide source-based answers. It is also more flexible than retraining an LLM whenever new documents are added.

10. Common Use Cases

RAG is commonly used for **document question answering, enterprise search, customer support, research assistants, legal and policy search, educational assistants, knowledge-base chatbots, and PDF-based AI assistants**.

11. Conclusion

RAG connects the knowledge retrieval capability of a search system with the language generation capability of an LLM. Its core idea is simple: **retrieve relevant information, provide it as context, and generate a grounded answer**. A reliable RAG system depends on good document processing, chunking, embeddings, retrieval, and prompt design.